Language Identification via Character N-Grams

Diego Besada Rodríguez

[My Code] [Models on Hugging Face] [Template]

October 2026

1 Objectives and Dataset

Language identification (LangID) is a fundamental initial filter in modern multilingual NLP pipelines. The objective of this laboratory is to design, implement, and evaluate a lightweight statistical language detector inspired by Google's Compact Language Detector v3 (CLD3) [2]. The system identifies the language of short sentences by projecting character n-gram distributions into a fixed-dimensional feature space using the hashing trick, subsequently classifying them through a Multi-Layer Perceptron (MLP).

We train and evaluate our classifier on the **Tatoeba** sentence corpus [3], which comprises over 13 million crowdsourced sentences spanning 429 languages. Because processing the full dataset (758 MB) is memory-prohibitive, we filter the collection using a lazy line-by-line streaming generator. We select a balanced subset of eight representative target languages spanning diverse scripts and typologies: Arabic (`ara`), Mandarin Chinese (`cmn`), Danish (`dan`), English (`eng`), French (`fra`), Japanese (`jpn`), Korean (`kor`), and Swedish (`swe`). Each class is capped at a maximum of 25,000 sentences (with Korean yielding 15,892 sentences), forming a consolidated experimental dataset of 190,892 samples stratified into an 80% training split (152,713 samples) and a 20% validation split (38,179 samples).

2 Method and Program Structure

2.1 Summary of Google's CLD3 Architecture and Feature Extraction

Google's CLD3 [2] is a neural model designed for rapid in-browser language detection. It extracts character unigrams, bigrams, and trigrams ($1 \leq n \leq 3$) from lowercased text. Because the raw vocabulary of character combinations across hundreds of writing systems exceeds millions of dimensions (in Tatoeba alone, we identified 11,728 unigrams, 548,380 bigrams, and 2,362,711 trigrams), CLD3 maps n-grams into bounded hash buckets via the *hashing trick* ($h(s) \pmod M$). In CLD3, each bucket corresponds to an embedding vector. The model computes the mean of these embeddings weighted by their relative frequencies, passes the resulting representation through a feed-forward layer (typically 16 to 64 hidden nodes) with ReLU activations, and outputs softmax logits across language classes.

To illustrate how character n-gram counts represent raw text as numerical vectors, the notebook extracts all unique subwords from the toy word *"banana"* up to length 2: unigrams (a, b, n) and bigrams (an, ba, na). We then count the exact occurrences of these six features across nine sample sentences from Tatoeba (three each for English, French, and Swedish). As shown in Table 1, each sentence becomes a row vector of raw counts paired with its language label, creating a sparse signature that a linear classifier can separate:

Sentence ID	a	b	n	an	ba	na	Target Language
1276	0	0	1	0	0	0	eng
1277	1	0	0	0	0	0	eng
1280	3	1	2	1	0	0	eng
1115	8	0	8	1	0	0	fra
1279	1	0	1	0	0	0	fra
1441	4	1	6	1	0	0	fra
337413	4	0	1	1	0	0	swe
341910	5	2	2	0	1	0	swe
341938	2	0	2	1	0	0	swe

Table 1: Toy feature count matrix X using the unique n-grams of *"banana"* ($n \leq 2$) evaluated on 9 Tatoeba sample sentences.

2.2 Our Architecture vs. Google CLD3

Our implementation uses deterministic MD5 hashing (via Marcus Klang's 64-bit integer generator) to guarantee reproducibility across operating systems. We allocate prime hash moduli bounds: $M_{\text{uni}} = 521$, $M_{\text{bi}} = 1031$, and $M_{\text{tri}} = 1031$. To eliminate inter-order hash collisions, we shift bigram buckets by 521 and trigram buckets by $521 + 1031 = 1552$, generating a unified feature dimension of $D = 2,583$.

While our pipeline shares CLD3's n-gram extraction, relative frequency normalization, and hashing mechanism, its network architecture differs in two fundamental ways:

- **Embedding Layer vs. Direct Sparse Projection:** CLD3 learns a dense embedding matrix $\mathbf{E} \in \mathbb{R}^{M \times d}$ where individual hash codes pull d-dimensional continuous vectors that are averaged together. In contrast, our model feeds the normalized frequency vector $\mathbf{x} \in \mathbb{R}^{2583}$ directly into a linear projection layer $\mathbf{W}_1 \in \mathbb{R}^{2583 \times 50}$.
- **Network Topology:** Our neural architecture consists of a 2-layer perceptron: $\mathbf{h} = \text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$ with 50 hidden neurons, followed by an unactivated output layer $\mathbf{z} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2 \in \mathbb{R}^8$, leaving log-softmax computation to the loss function.

2.3 Architectural and Procedural Differences: Scikit-Learn vs. PyTorch

We implement this classifier under both scikit-learn (`MLPClassifier`) and PyTorch (`nn.Sequential`):

1. **Encapsulation vs. Explicit Mechanics:** In scikit-learn, the call `clf.fit(X_train, y_train)` encapsulates batch sampling, forward pass evaluation, cross-entropy loss computation, analytical backpropagation, and weight updates under Adam. In PyTorch, these mechanics are coded explicitly inside a nested epoch-and-batch loop: zeroing prior gradients (`optimizer.zero_grad()`), forward inference (`model(X_batch)`), backward autograd graph traversal (`loss.backward()`), and parameter updates (`optimizer.step()`).
2. **Data Ingestion and Memory Layout:** Scikit-learn consumes static in-memory NumPy arrays (`float32`). PyTorch abstracts ingestion through `TensorDataset` and `DataLoader`, streaming mini-batches of size 32 with dynamic shuffling at each epoch, enabling efficient GPU offloading and bounded memory consumption.
3. **Evaluation and Output Types:** Scikit-learn's `predict()` directly outputs predicted discrete class indices. PyTorch outputs unnormalized logits `z`; we explicitly compute predictive class probabilities via `torch.softmax(logits, dim1)`= and obtain class indices with `torch.argmax()` under a `torch.no_grad()` context.

3 Results

Both implementations demonstrate near-perfect classification performance across the eight languages, as shown in Table 2. The PyTorch model was trained for 7 epochs using Adam (lr = 0.001), converging from an initial training loss of 0.1585 down to 0.0092 (validation loss 0.0288).

Framework	Overall Accuracy	Micro $\mathbf{F}_1$	Macro $\mathbf{F}_1$	Parameters ($D \to H \to C$)
Scikit-Learn (`MLPClassifier`)	99.10%	0.9910	0.9913	$2583 \to 50 \to 8$ (5 epochs)
PyTorch (`nn.Sequential`)	**99.22%**	**0.9922**	**0.9924**	$2583 \to 50 \to 8$ (7 epochs)

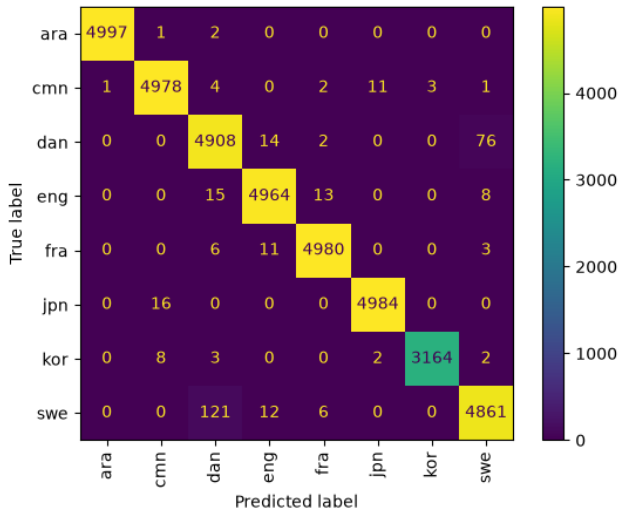
Table 2: Global performance comparison on the Tatoeba validation set (38,179 sentences).

The confusion matrices in Figure 1 highlight identical, linguistically expected classification behaviors:

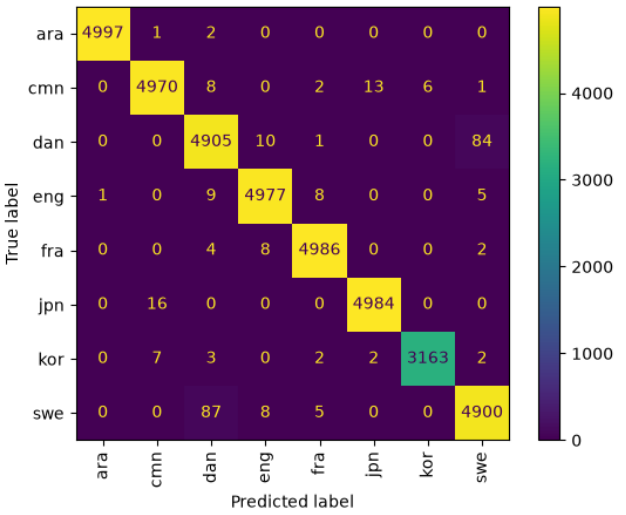
- **Distant Scripts:** Languages with distinct scripts (`ara`, `cmn`, `jpn`, `kor`) achieve near 100% precision and recall, as their character unigram buckets exhibit zero lexical overlap with Latin alphabets.
- **Orthographic and Typological Proximity:** The primary residual misclassifications occur strictly between **Danish** (`dan`) and **Swedish** (`swe`), which share high proportions of cognates and identical subword character n-grams. Danish yields 0.98 F_1 with small leakages into Swedish (and vice-versa).

When tested on out-of-domain short sentences (*"Salut les gars !"*, *"Hejsan grabbar!"*, *"Hello guys!"*, *"Hejsan tjejer!"*), both models correctly identify French, Swedish, English, and Danish respectively.

The serialized models, vectorizers, and PyTorch checkpoint weights (`nld.pth`, `nld.joblib`, `nld_vectorizer.joblib`, `nld_label_encoder.joblib`) are deposited and publicly available on Hugging Face:
https://huggingface.co/dbsda/EDAN20-Lab4-LanguageDetector



(a) Scikit-Learn confusion matrix.



(b) PyTorch confusion matrix.

Figure 1: Normalized confusion matrices for Scikit-Learn and PyTorch classifiers across 8 languages.

4 Conclusion and Future Improvements

This laboratory illustrates that character n-grams combined with feature hashing provide an exceptionally fast, highly accurate mechanism for text classification without requiring word-level tokenization or morphological parsers. Transitioning from scikit-learn to PyTorch preserves predictive accuracy (99.10% vs. 99.22%) while providing fine-grained control over batch scheduling, gradient backpropagation, and loss inspection.

Following the large-scale findings from Google Research in *Building Machine Translation Systems for the Next Thousand Languages* [1], pure n-gram CLD3-style models [2] suffer from well-documented error pathologies when scaled to web data:

- **Collision Mitigation via Increased Moduli and Multi-Hashing:** Our analysis revealed significant hash collisions (e.g., an expected average of 2,291 trigrams competing for each of the 1,031 buckets). Expanding the bucket dimension M or implementing double hashing would drastically reduce feature interference.
- **Semi-Supervised Pretraining (SSLID):** As highlighted by Bapna et al. [1], pure n-gram bag models fail on noisy, short, or code-switched web content. Replacing surface bag-of-n-grams with pretrained masked sequence representations (e.g., MASS/Transformer encoders) captures semantic context and sentence syntax.
- **Document-Consistency Filtering:** Short isolated sentences often contain insufficient character evidence. Enforcing document-level majority voting (discarding sentences that conflict with the predominant language of the parent document) reliably removes false positives on low-resource and noisy text.

References

[1] Ankur Bapna, Isaac Caswell, Julia Kreutzer, Orhan Firat, Daan van Esch, Aditya Siddhant, Mengmeng Niu, Pallavi Baljekar, Xavier Garcia, Wolfgang Macherey, et al. Building machine translation systems for the next thousand languages. *arXiv preprint arXiv:2205.03983*, 2022.

[2] Google. CLD3: Compact Language Detector v3. https://github.com/google/cld3, 2016.

[3] Tatoeba Project. Tatoeba: Open-source collection of sentences and translations. https://tatoeba.org/, 2026.